DeepL
Subscribe to DeepL Pro to translate larger documents.
Visit www.DeepL.com/pro for more information.

CHƯƠNG 3
Phân loại

Trong Chương 1, tôi đã đề cập rằng các tác vụ học có giám sát phổ biến nhất là hồi quy (dự đoán giá trị) và phân loại (dự đoán lớp). Trong Chương 2, chúng ta đã tìm hiểu về một tác vụ hồi quy, dự đoán giá trị nhà ở, bằng cách sử dụng các thuật toán khác nhau như hồi quy tuyến tính, cây quyết định và rừng ngẫu nhiên (sẽ được giải thích chi tiết hơn trong các chương sau). Bây giờ, chúng ta sẽ chuyển sự chú ý sang các hệ thống phân loại.

MNIST

Trong chương này, chúng ta sẽ sử dụng tập dữ liệu MNIST, bao gồm 70.000 hình ảnh nhỏ của các chữ số được viết tay bởi học sinh trung học và nhân viên Cục Thống kê Hoa Kỳ. Mỗi hình ảnh đều được gắn nhãn với chữ số mà nó đại diện. Bộ dữ liệu này đã được nghiên cứu rất nhiều đến mức thường được gọi là "hello world" của học máy: mỗi khi ai đó phát triển một thuật toán phân loại mới, họ đều tò mò muốn xem thuật toán đó sẽ hoạt động như thế nào trên MNIST, và bất kỳ ai học về học máy đều sẽ phải làm việc với bộ dữ liệu này sớm hay muộn.

Scikit-Learn cung cấp nhiều hàm hỗ trợ để tải xuống các bộ dữ liệu phổ biến. MNIST là một trong số đó. Đoạn mã sau đây tải bộ dữ liệu MNIST từ OpenML.org:[1]

```
from sklearn.datasets import fetch_openml

mnist = fetch_openml('mnist_784', as_frame=False)
```

1 Theo mặc định, Scikit-Learn lưu trữ các bộ dữ liệu đã tải xuống trong một thư mục có tên là *scikit_learn_data* trong thư mục home của bạn.

Gói `sklearn.datasets` chủ yếu chứa ba loại hàm: các hàm `fetch_*` như `fetch_openml()` để tải xuống các bộ dữ liệu thực tế, các hàm `load_*` để tải các bộ dữ liệu mẫu nhỏ đi kèm với Scikit-Learn (do đó không cần tải xuống từ internet), và các hàm `make_*` để tạo ra các bộ dữ liệu giả, hữu ích cho việc kiểm thử. Các tập dữ liệu được tạo ra thường được trả về dưới dạng một bộ `(X, y)` chứa dữ liệu đầu vào và các mục tiêu, cả hai đều là các mảng NumPy. Các tập dữ liệu khác được trả về dưới dạng các đối tượng `sklearn.utils.Bunch`, là các từ điển mà các mục của chúng cũng có thể được truy cập dưới dạng các thuộc tính. Chúng thường chứa các mục sau:

`"DESCR"`
: Mô tả về tập dữ liệu

`"data"`
: Dữ liệu đầu vào, thường là mảng NumPy 2D

`"target"`
: Các nhãn, thường là một mảng NumPy 1 chiều

Hàm `fetch_openml()` hơi khác thường vì theo mặc định, nó trả về dữ liệu đầu vào dưới dạng DataFrame của Pandas và các nhãn dưới dạng Series của Pandas (trừ khi tập dữ liệu là thưa thớt). Tuy nhiên, tập dữ liệu MNIST chứa hình ảnh, và DataFrame không phải là lựa chọn lý tưởng cho mục đích này, do đó nên đặt `as_frame=False` để nhận dữ liệu dưới dạng mảng NumPy thay thế. Hãy xem xét các mảng này:

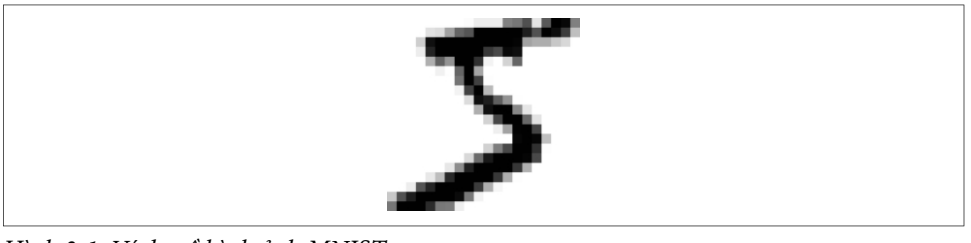
```
>>> X, y = mnist.data, mnist.target
>>> X
array([[0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       ...,
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0]])
>>> X.shape
(70000, 784)
>>> y
array(['5', '0', '4', ..., '4', '5', '6'], dtype=object)
>>> y.shape
(70000,)
```

Có 70.000 hình ảnh, và mỗi hình ảnh có 784 đặc trưng. Điều này là do mỗi hình ảnh có kích thước 28 × 28 pixel, và mỗi đặc trưng đơn giản đại diện cho độ sáng của một pixel, từ 0 (trắng) đến 255 (đen). Hãy cùng xem qua một chữ số trong tập dữ liệu (Hình 3-1). Tất cả những gì chúng ta cần làm là lấy vectơ đặc trưng của một đối tượng, định dạng lại thành mảng 28 × 28 và hiển thị nó bằng hàm `imshow()` của Matplotlib. Chúng ta sử dụng `cmap="binary"` để có được bản đồ màu xám, trong đó 0 là màu trắng và 255 là màu đen:

```
import matplotlib.pyplot as plt

def plot_digit(image_data):
    image = image_data.reshape(28, 28)
    plt.imshow(image, cmap="binary") plt.axis("off")

some_digit = X[0]
plot_digit(some_digit)
plt.show()
```



Hình 3-1. Ví dụ về hình ảnh MNIST

Hình này trông giống số 5, và quả thực đó chính là thông tin mà nhãn dữ liệu cho biết:

```
>>> y[0]
'5'
```

Để bạn có thể cảm nhận được mức độ phức tạp của tác vụ phân loại, Hình 3-2 trình bày thêm một số hình ảnh từ tập dữ liệu MNIST. Có sự đa dạng khá lớn về hình dạng các chữ số. Tuy nhiên, các hình ảnh này rõ nét, được căn giữa tốt, không bị xoay quá nhiều và các chữ số đều có kích thước gần như nhau: tập dữ liệu này sẽ không cần nhiều bước tiền xử lý (các tập dữ liệu thực tế thường không thân thiện như vậy).

Nhưng khoan đã! Bạn nên luôn tạo một tập kiểm tra và tách riêng nó ra trước khi kiểm tra dữ liệu kỹ lưỡng. Bộ dữ liệu MNIST được trả về bởi hàm `fetch_openml()` thực tế đã được chia sẵn thành tập huấn luyện (60.000 hình ảnh đầu tiên) và tập kiểm tra (10.000 hình ảnh cuối cùng):[2]

```
X_train, X_test, y_train, y_test = X[:60000], X[60000:], y[:60000], y[60000:]
```

Bộ dữ liệu huấn luyện đã được xáo trộn sẵn cho chúng ta, điều này rất tốt vì nó đảm bảo rằng tất cả các phần (fold) trong quá trình kiểm tra chéo sẽ tương tự nhau (chúng ta không muốn một phần nào đó thiếu một số chữ số). Hơn nữa, một số thuật toán học máy rất nhạy cảm với thứ tự của các bản ghi huấn luyện, và chúng sẽ hoạt động kém hiệu quả nếu gặp nhiều bản ghi tương tự liên tiếp. Việc xáo trộn bộ dữ liệu đảm bảo rằng điều này sẽ không xảy ra.[3]

2 Các tập dữ liệu được trả về bởi hàm `fetch_openml()` không phải lúc nào cũng được trộn ngẫu nhiên hoặc chia nhỏ.

3 Việc xáo trộn có thể là một ý tưởng tồi trong một số bối cảnh — ví dụ, nếu bạn đang làm việc với dữ liệu chuỗi thời gian (như giá cổ phiếu hoặc điều kiện thời tiết). Chúng ta sẽ tìm hiểu điều này trong Chương 13.



Hình 3-2. Các chữ số từ tập dữ liệu MNIST

Huấn luyện một bộ phân loại nhị phân

Hãy đơn giản hóa vấn đề trước mắt và chỉ cố gắng nhận diện một chữ số — ví dụ, số 5. "Bộ phát hiện số 5" này sẽ là một ví dụ về *mô hình phân loại nhị phân*, có khả năng phân biệt giữa chỉ hai lớp: 5 và không phải 5. Đầu tiên, chúng ta sẽ tạo các vectơ mục tiêu cho tác vụ phân loại này:

```
y_train_5 = (y_train == '5') # True cho tất cả các số 5, False cho tất cả các chữ số khác
y_test_5 = (y_test == '5')
```

Bây giờ chúng ta hãy chọn một mô hình phân loại và huấn luyện nó. Một điểm khởi đầu tốt là sử dụng mô hình phân loại *theo phương pháp Gradient Descent ngẫu nhiên* (SGD, hay GD ngẫu nhiên), thông qua lớp `SGDClassifier` của Scikit-Learn. Mô hình phân loại này có khả năng xử lý các tập dữ liệu rất lớn một cách hiệu quả. Điều này một phần là do SGD xử lý các bản ghi huấn luyện một cách độc lập, từng bản ghi một, điều này cũng khiến SGD rất phù hợp cho việc học trực tuyến, như bạn sẽ thấy sau này. Hãy tạo một đối tượng `SGDClassifier` và huấn luyện nó trên toàn bộ tập huấn luyện:

```
from sklearn.linear_model import SGDClassifier
```

```
sgd_clf = SGDClassifier(random_state=42)
sgd_clf.fit(X_train, y_train_5)
```

Giờ đây, chúng ta có thể sử dụng nó để nhận diện các hình ảnh của số 5:

```
>>> sgd_clf.predict([some_digit]) array([
True])
```

Bộ phân loại đoán rằng hình ảnh này đại diện cho số 5 (`True`). Có vẻ như nó đã đoán đúng trong trường hợp cụ thể này! Bây giờ, hãy đánh giá hiệu suất của mô hình này.

Các chỉ số đánh giá hiệu suất

Việc đánh giá một bộ phân loại thường khó hơn nhiều so với việc đánh giá một bộ hồi quy, vì vậy chúng ta sẽ dành phần lớn chương này để thảo luận về chủ đề này. Có rất nhiều chỉ số hiệu suất, vì vậy hãy pha thêm một tách cà phê và sẵn sàng để tìm hiểu một loạt các khái niệm và từ viết tắt mới!

Đo lường độ chính xác bằng phương pháp kiểm chứng chéo

Một cách tốt để đánh giá mô hình là sử dụng kiểm tra chéo, giống như bạn đã làm trong Chương 2. Hãy sử dụng hàm `cross_val_score()` để đánh giá mô hình `SGDClassifier` của chúng ta, bằng cách sử dụng kiểm tra chéo k-fold với ba phần. Hãy nhớ rằng kiểm định chéo k-fold có nghĩa là chia tập huấn luyện thành *k* phần (trong trường hợp này là ba), sau đó huấn luyện mô hình *k* lần, mỗi lần giữ lại một phần khác nhau để đánh giá (xem Chương 2):

```
>>> from sklearn.model_selection import cross_val_score
>>> cross_val_score(sgd_clf, X_train, y_train_5, cv=3, scoring="accuracy") array([0.95035,
0.96035, 0.9604 ])
```

Wow! Độ chính xác trên 95% (tỷ lệ dự đoán chính xác) trên tất cả các tập dữ liệu kiểm tra chéo? Kết quả này thật ấn tượng, phải không? Tuy nhiên, trước khi bạn quá phấn khích, hãy xem xét một mô hình phân loại giả (dummy classifier) chỉ phân loại mọi hình ảnh vào lớp xuất hiện nhiều nhất, trong trường hợp này là lớp âm (tức là không phải số 5):

```
from sklearn.dummy import DummyClassifier

dummy_clf = DummyClassifier() dummy_clf.fit(X_train,
y_train_5)
print(any(dummy_clf.predict(X_train))) # in ra False: không phát hiện thấy số 5 nào
```

Bạn có đoán được độ chính xác của mô hình này không? Hãy cùng tìm hiểu:

```
>>> cross_val_score(dummy_clf, X_train, y_train_5, cv=3, scoring="accuracy")
array([0.90965, 0.90965, 0.90965])
```

Đúng vậy, độ chính xác của nó vượt quá 90%! Điều này đơn giản là do chỉ khoảng 10% hình ảnh là số 5, nên nếu bạn luôn đoán rằng một hình ảnh *không phải* là số 5, bạn sẽ đúng khoảng 90% số lần. Thậm chí còn hơn cả Nostradamus.

Điều này chứng minh lý do tại sao độ chính xác thường không phải là thước đo hiệu suất được ưa chuộng cho các bộ phân loại, đặc biệt là khi bạn đang xử lý *các tập dữ liệu lệch* (tức là khi một số lớp xuất hiện thường xuyên hơn nhiều so với các lớp khác). Một cách tốt hơn nhiều để đánh giá hiệu suất của bộ phân loại là xem xét *ma trận nhầm lẫn* (CM).

Thực hiện kiểm tra chéo

Đôi khi, bạn sẽ cần kiểm soát quá trình kiểm chứng chéo nhiều hơn so với những gì Scikit-Learn cung cấp sẵn. Trong những trường hợp này, bạn có thể tự triển khai kiểm chứng chéo. Đoạn mã sau đây thực hiện chức năng gần giống như hàm `cross_val_score()` của Scikit-Learn và hiển thị kết quả tương tự:

```
from sklearn.model_selection import StratifiedKFold from
sklearn.base import clone

skfolds = StratifiedKFold(n_splits=3) # thêm shuffle=True nếu tập dữ liệu
                                      # chưa được xáo trộn
for train_index, test_index in skfolds.split(X_train, y_train_5): clone_clf
    = clone(sgd_clf)
    X_train_folds = X_train[train_index]
    y_train_folds = y_train_5[train_index]
    X_test_fold = X_train[test_index] y_test_fold
    = y_train_5[test_index]

    clone_clf.fit(X_train_folds, y_train_folds)
    y_pred = clone_clf.predict(X_test_fold)
    n_correct = sum(y_pred == y_test_fold)
    print(n_correct / len(y_pred)) # in ra 0.95035, 0.96035 và 0.9604
```

Lớp `StratifiedKFold` thực hiện phương pháp lấy mẫu phân tầng (như đã giải thích trong Chương 2) để tạo ra các nhóm mẫu (folds) chứa tỷ lệ đại diện của từng lớp. Tại mỗi lần lặp, mã nguồn sẽ tạo ra một bản sao của mô hình phân loại, huấn luyện bản sao đó trên các nhóm mẫu huấn luyện và đưa ra dự đoán trên nhóm mẫu kiểm tra. Sau đó, nó đếm số lượng dự đoán chính xác và trả về tỷ lệ dự đoán chính xác.

Ma trận nhầm lẫn

Ý tưởng chung của ma trận nhầm lẫn là đếm số lần các mẫu thuộc lớp A bị phân loại nhầm thành lớp B, cho tất cả các cặp A/B. Ví dụ, để biết số lần mô hình phân loại nhầm các hình ảnh số 8 thành số 0, bạn sẽ xem hàng số 8, cột số 0 của ma trận nhầm lẫn.

Để tính toán ma trận nhầm lẫn, trước tiên bạn cần có một tập hợp các dự đoán để có thể so sánh với các mục tiêu thực tế. Bạn có thể thực hiện dự đoán trên tập dữ liệu kiểm tra, nhưng tốt nhất là nên để nguyên tập dữ liệu đó cho đến lúc này (hãy nhớ rằng bạn chỉ nên sử dụng tập dữ liệu kiểm tra vào giai đoạn cuối cùng của dự án, khi đã có mô hình phân loại sẵn sàng để triển khai). Thay vào đó, bạn có thể sử dụng hàm `` `cross_val_predict()` ``:

```
from sklearn.model_selection import cross_val_predict

y_train_pred = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3)
```

Giống như hàm `cross_val_score()`, `cross_val_predict()` thực hiện kiểm định chéo k-fold, nhưng thay vì trả về các điểm đánh giá, nó trả về các dự đoán được thực hiện trên từng tập kiểm tra. Điều này có nghĩa là bạn nhận được một dự đoán "sạch" cho mỗi bản ghi trong tập huấn luyện (khi nói "sạch", tôi muốn nói "ngoài mẫu": mô hình đưa ra dự đoán trên dữ liệu mà nó chưa từng gặp trong quá trình huấn luyện).

Bây giờ bạn đã sẵn sàng để lấy ma trận nhầm lẫn bằng hàm `confusion_matrix()`. Chỉ cần truyền vào các lớp mục tiêu (`y_train_5`) và các lớp dự đoán (`y_train_pred`):

```
>>> from sklearn.metrics import confusion_matrix
>>> cm = confusion_matrix(y_train_5, y_train_pred)
>>> cm
array([[53892,     687],
       [ 1891, 3530]])
```

Mỗi hàng trong ma trận nhầm lẫn đại diện cho một *lớp thực tế*, trong khi mỗi cột đại diện cho một *lớp dự đoán*. Hàng đầu tiên của ma trận này xem xét các hình ảnh không phải số 5 (*lớp âm tính*): 53.892 trong số đó được phân loại chính xác là không phải số 5 (được gọi là *âm tính thực*), trong khi 687 hình ảnh còn lại bị phân loại sai là số 5 (*dương tính giả*, còn được gọi là *lỗi loại I*). Hàng thứ hai xem xét các hình ảnh thuộc lớp 5 (*lớp dương*): 1.891 hình ảnh bị phân loại sai thành không phải lớp 5 (*kết quả âm giả*, còn gọi là *lỗi loại II*), trong khi 3.530 hình ảnh còn lại được phân loại chính xác là lớp 5 (*kết quả dương đúng*). Một bộ phân loại hoàn hảo sẽ chỉ có dương tính thật và âm tính thật, do đó ma trận nhầm lẫn của nó sẽ chỉ có các giá trị khác 0 trên đường chéo chính (từ trên cùng bên trái xuống dưới cùng bên phải):

```
>>> y_train_perfect_predictions = y_train_5 # giả sử chúng ta đã đạt đến mức hoàn hảo
>>> confusion_matrix(y_train_5, y_train_perfect_predictions)
array([[54579,       0],
       [    0, 5421]])
```

Ma trận nhầm lẫn cung cấp cho bạn rất nhiều thông tin, nhưng đôi khi bạn có thể muốn một thước đo ngắn gọn hơn. Một thước đo thú vị để xem xét là độ chính xác của các dự đoán dương; đây được gọi là độ *chính xác* của bộ phân loại (Phương trình 3-1).

Phương trình 3-1. Độ chính xác

$$\text{độ chính xác} = \frac{TP}{TP + FP}$$

TP là số kết quả dương tính đúng, còn *FP* là số kết quả dương tính sai.

Giờ hãy xem xét một mô hình chỉ đưa ra dự đoán dương tính khi nó cực kỳ tự tin. Hãy đẩy vấn đề này đến mức cực đoan và giả sử rằng nó luôn đưa ra các dự đoán âm tính

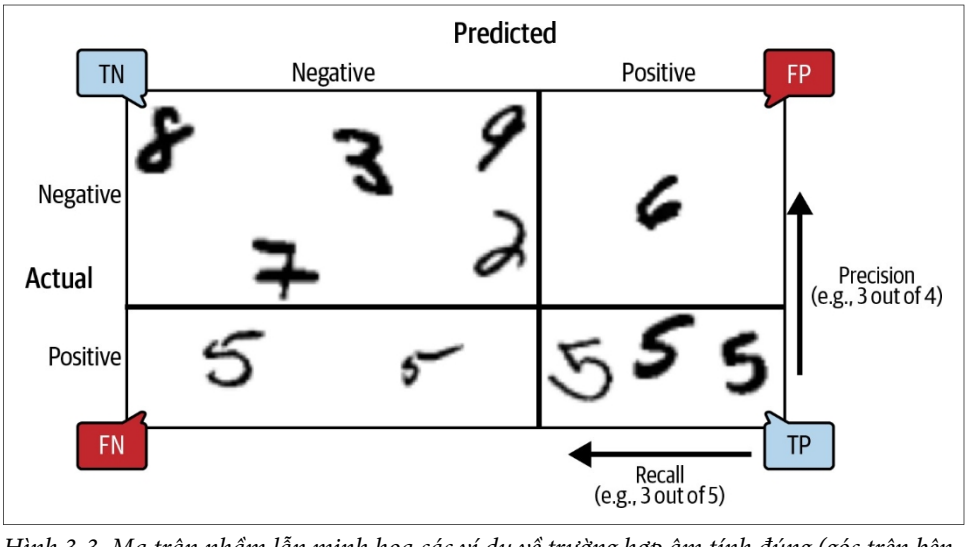
, ngoại trừ một dự đoán dương duy nhất đối với trường hợp mà nó tự tin nhất. Nếu dự đoán duy nhất này là chính xác, thì bộ phân loại sẽ có độ chính xác 100% (độ chính xác = 1/1 = 100%). Rõ ràng, một bộ phân loại như vậy sẽ không hữu ích lắm, vì nó sẽ bỏ qua tất cả các trường hợp dương tính ngoại trừ một trường hợp duy nhất. Vì lý do này, độ chính xác thường được sử dụng cùng với một chỉ số khác gọi là *độ thu hồi (recall)*, còn được gọi là *độ nhạy (sensitivity)* hoặc *tỷ lệ dương tính thực* (TPR): đây là tỷ lệ các trường hợp dương tính được bộ phân loại phát hiện chính xác (Phương trình 3-2).

Phương trình 3-2. Tỷ lệ thu hồi

$$\text{recall} = \frac{TP}{TP + FN}$$

FN, tất nhiên, là số kết quả âm tính giả.

Nếu bạn cảm thấy bối rối về ma trận nhầm lẫn, Hình 3-3 có thể giúp bạn.



Hình 3-3. Ma trận nhầm lẫn minh họa các ví dụ về trường hợp âm tính đúng (góc trên bên trái), dương tính giả (góc trên bên phải), âm tính giả (góc dưới bên trái) và dương tính đúng (góc dưới bên phải)

Độ chính xác và độ thu hồi

Scikit-Learn cung cấp một số hàm để tính toán các chỉ số của mô hình phân loại, bao gồm độ chính xác và độ nhạy:

```
>>> from sklearn.metrics import precision_score, recall_score
>>> precision_score(y_train_5, y_train_pred) # == 3530 / (687 + 3530)
np.float64(0.8370879772350012)
>>> recall_score(y_train_5, y_train_pred) # == 3530 / (1891 + 3530)
np.float64(0.6511713705958311)
```

Giờ đây, mô hình 5-detector của chúng ta không còn ấn tượng như khi chúng ta xem xét độ chính xác của nó. Khi mô hình khẳng định một hình ảnh là số 5, nó chỉ đúng 83,7% số lần. Hơn nữa, nó chỉ phát hiện được 65,1% các trường hợp số 5.

Thường thì việc kết hợp độ chính xác và độ nhạy thành một chỉ số duy nhất gọi là *điểm F_1* rất thuận tiện, đặc biệt khi bạn cần một chỉ số duy nhất để so sánh hai bộ phân loại. Chỉ số F_1 là *trung bình hài hòa* của độ chính xác và độ thu hồi (Phương trình 3-3). Trong khi trung bình thông thường coi tất cả các giá trị là như nhau, thì trung bình hài hòa lại cho trọng số lớn hơn nhiều đối với các giá trị thấp. Do đó, bộ phân loại sẽ chỉ đạt được chỉ số F_1 cao nếu cả độ thu hồi và độ chính xác đều cao.

Phương trình 3-3. Điểm F_1

$$F_1 = \frac{2}{\frac{1}{\text{độ chính xác}} + \frac{1}{\text{độ thu hồi}}} = 2 \times \frac{\text{độ chính xác} \times \text{độ thu hồi}}{\text{độ chính xác} + \text{độ bao quát}} = \frac{TP}{TP + \frac{FN + FP}{2}}$$

Để tính điểm F_1, chỉ cần gọi hàm `f1_score()`:

```
>>> from sklearn.metrics import f1_score
>>> f1_score(y_train_5, y_train_pred)
0.7325171197343846
```

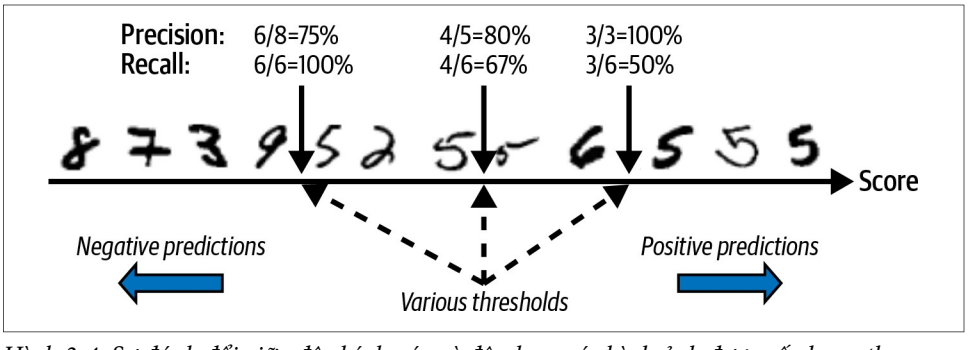
Điểm F_1 ưu tiên các bộ phân loại có độ chính xác và độ thu hồi tương tự nhau. Điều này không phải lúc nào cũng là điều bạn mong muốn: trong một số trường hợp, bạn chủ yếu quan tâm đến độ chính xác, còn trong những trường hợp khác, bạn lại thực sự quan tâm đến độ thu hồi. Ví dụ, nếu bạn huấn luyện một mô hình phân loại để phát hiện các video an toàn cho trẻ em, bạn có thể sẽ ưa chuộng mô hình loại bỏ nhiều video tốt (độ thu hồi thấp) nhưng chỉ giữ lại những video an toàn (độ chính xác cao), thay vì mô hình có độ thu hồi cao hơn nhiều nhưng lại để lọt một vài video thực sự xấu vào sản phẩm của bạn (trong những trường hợp như vậy, bạn thậm chí có thể muốn thêm một quy trình kiểm tra thủ công để xem xét lựa chọn video của mô hình). Mặt khác, giả sử bạn huấn luyện một mô hình phân loại để phát hiện kẻ trộm trong hình ảnh giám sát: có lẽ không sao nếu mô hình chỉ có độ chính xác 30% miễn là nó có độ nhạy 99%. Dĩ nhiên, nhân viên an ninh sẽ nhận được một vài cảnh báo sai, nhưng hầu hết kẻ trộm sẽ bị bắt. Tương tự, chẩn đoán y tế thường yêu cầu độ nhạy cao để tránh bỏ sót bất kỳ điều gì quan trọng. Các kết quả dương tính giả có thể được loại trừ thông qua các xét nghiệm y tế tiếp theo.

Thật không may, bạn không thể có được cả hai: tăng độ chính xác sẽ làm giảm độ thu hồi, và ngược lại. Điều này được gọi là *sự đánh đổi giữa độ chính xác và độ thu hồi*.

Sự đánh đổi giữa độ chính xác và độ nhạy

Để hiểu sự đánh đổi này, hãy xem cách `SGDClassifier` đưa ra quyết định phân loại. Đối với mỗi mẫu, nó tính toán một điểm số dựa trên *hàm quyết định*. Nếu điểm số đó lớn hơn ngưỡng, nó sẽ gán mẫu đó vào lớp dương; ngược lại, nó sẽ gán vào lớp âm. Hình 3-4 hiển thị một số chữ số được sắp xếp

từ điểm số thấp nhất ở bên trái đến điểm số cao nhất ở bên phải. Giả sử *ngưỡng quyết định* được đặt tại mũi tên ở giữa (giữa hai số 5): bạn sẽ thấy 4 trường hợp dương tính đúng (số 5 thực sự) ở bên phải ngưỡng đó, và 1 trường hợp dương tính giả (thực ra là số 6). Do đó, với ngưỡng đó, độ chính xác là 80% (4 trên 5). Tuy nhiên, trong số 6 chữ số 5 thực sự, mô hình phân loại chỉ phát hiện được 4, nên độ nhạy là 67% (4 trên 6). Nếu bạn nâng ngưỡng (di chuyển nó sang mũi tên bên phải), trường hợp dương tính giả (số 6) sẽ trở thành trường hợp âm tính đúng, từ đó làm tăng độ chính xác (lên đến 100% trong trường hợp này), nhưng một trường hợp dương tính đúng sẽ trở thành trường hợp âm tính giả, làm giảm độ nhạy xuống còn 50%. Ngược lại, hạ ngưỡng sẽ làm tăng độ nhạy và giảm độ chính xác.



Hình 3-4. Sự đánh đổi giữa độ chính xác và độ nhạy: các hình ảnh được xếp hạng theo điểm số của bộ phân loại, và những hình ảnh có điểm số cao hơn ngưỡng quyết định đã chọn được coi là dương; ngưỡng càng cao, độ nhạy càng thấp, nhưng (nói chung) độ chính xác càng cao

Thay vì gọi phương thức `predict()` của mô hình phân loại, bạn có thể gọi phương thức `decision_function()` của nó, phương thức này trả về điểm số cho từng mẫu. Sau đó, bạn có thể sử dụng bất kỳ ngưỡng nào bạn muốn để đưa ra dự đoán dựa trên các điểm số đó:

```
>>> y_scores = sgd_clf.decision_function([some_digit])
>>> y_scores
array([2164.22030239])
>>> threshold = 0
>>> y_some_digit_pred = (y_scores > threshold)
>>> y_some_digit_pred array([
True])
```

`SGDClassifier` sử dụng ngưỡng bằng 0, do đó đoạn mã trên trả về kết quả giống như phương thức `predict()` (tức là `True`). Hãy tăng ngưỡng lên:

```
>>> threshold = 3000
>>> y_some_digit_pred = (y_scores > threshold)
>>> y_some_digit_pred
array([False])
```

Điều này xác nhận rằng việc tăng ngưỡng sẽ làm giảm độ nhạy. Hình ảnh thực tế thể hiện số 5, và bộ phân loại phát hiện được nó khi ngưỡng là 0, nhưng lại bỏ sót khi ngưỡng được tăng lên 3.000.

Làm thế nào để quyết định nên sử dụng ngưỡng nào? Một lựa chọn là sử dụng hàm `cross_val_predict()` để lấy điểm số của tất cả các bản ghi trong tập huấn luyện, nhưng lần này chỉ định rằng bạn muốn trả về điểm số quyết định thay vì dự đoán:

```
y_scores = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3,
                             method="decision_function")
```

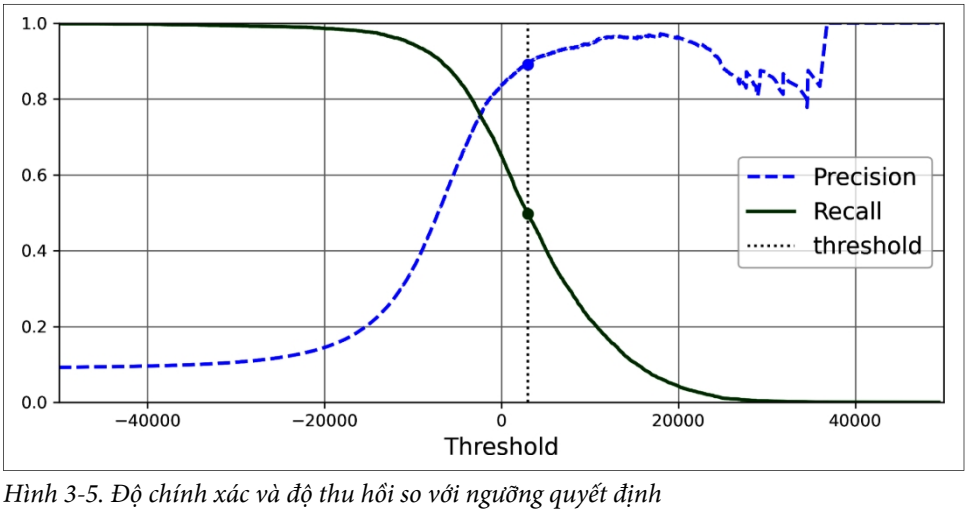
Với các điểm số này, hãy sử dụng hàm `precision_recall_curve()` để tính độ chính xác và độ thu hồi cho tất cả các ngưỡng có thể (hàm này thêm độ chính xác cuối cùng là 1 và độ thu hồi cuối cùng là 0, tương ứng với ngưỡng vô hạn):

```
from sklearn.metrics import precision_recall_curve

precisions, recalls, thresholds = precision_recall_curve(y_train_5, y_scores)
```

Cuối cùng, sử dụng Matplotlib để vẽ đồ thị độ chính xác và độ thu hồi theo giá trị ngưỡng (Hình 3-5). Hãy hiển thị ngưỡng 3.000 mà chúng ta đã chọn:

```
plt.plot(thresholds, precisions[:-1], "b--", label="Precision", linewidth=2)
plt.plot(thresholds, recalls[:-1], "g-", label="Recall", linewidth=2)
plt.vlines(threshold, 0, 1.0, "k", "dotted", label="threshold")
[...] # làm đẹp biểu đồ: thêm lưới, chú thích, trục tọa độ, nhãn và các vòng tròn
plt.show()
```



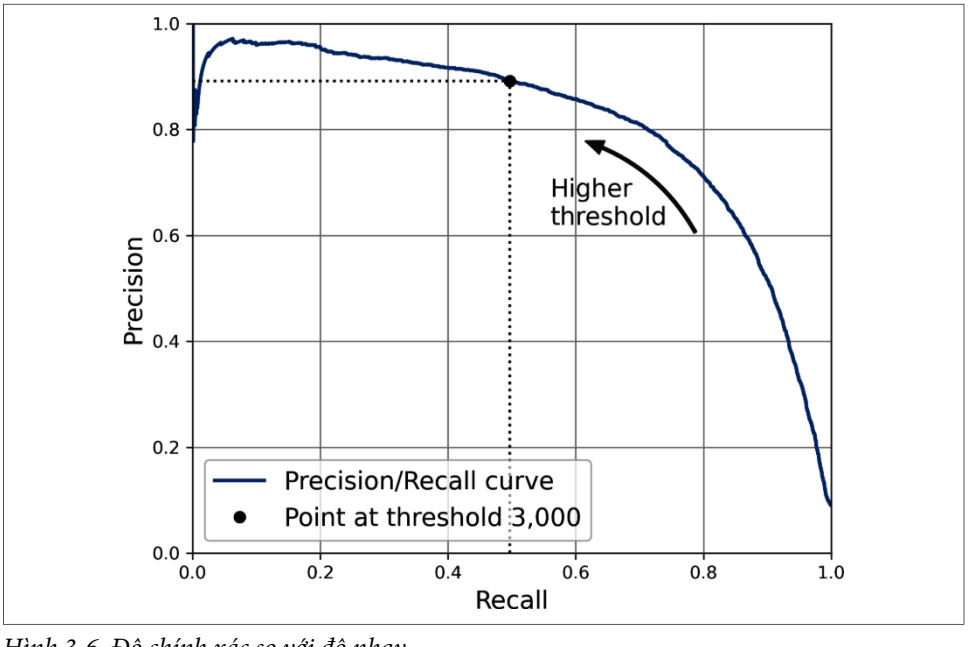
Hình 3-5. Độ chính xác và độ thu hồi so với ngưỡng quyết định



Bạn có thể thắc mắc tại sao đường cong độ chính xác lại gập ghềnh hơn đường cong độ thu hồi trong Hình 3-5. Lý do là độ chính xác đôi khi có thể giảm khi bạn tăng ngưỡng (mặc dù nói chung nó sẽ tăng). Để hiểu lý do, hãy xem lại Hình 3-4 và lưu ý điều gì xảy ra khi bạn bắt đầu từ ngưỡng trung tâm và dịch chuyển nó sang phải chỉ một đơn vị: độ chính xác giảm từ 4/5 (80%) xuống còn 3/4 (75%). Mặt khác, độ bao quát chỉ có thể giảm khi ngưỡng được tăng lên, điều này giải thích tại sao đường cong của nó trông mượt mà.

Ở giá trị ngưỡng này, độ chính xác gần 90% và độ thu hồi khoảng 50%. Một cách khác để chọn sự cân bằng tốt giữa độ chính xác và độ thu hồi là vẽ đồ thị độ chính xác trực tiếp so với độ thu hồi, như được thể hiện trong Hình 3-6 (cùng một ngưỡng được hiển thị):

```
plt.plot(recalls, precisions, linewidth=2, label="Đường cong độ chính xác/độ
nhạy") [...] # làm đẹp hình vẽ: thêm nhãn, lưới, chú thích, mũi tên và văn bản
plt.show()
```



Hình 3-6. Độ chính xác so với độ nhạy

Bạn có thể thấy rằng độ chính xác thực sự bắt đầu giảm mạnh khi độ thu hồi đạt khoảng 80%. Bạn có thể sẽ muốn chọn điểm cân bằng giữa độ chính xác và độ thu hồi ngay trước khi mức giảm đó xảy ra — ví dụ, khi độ thu hồi đạt khoảng 60%. Nhưng tất nhiên, sự lựa chọn này phụ thuộc vào dự án của bạn.

Giả sử bạn quyết định nhắm đến độ chính xác 90%. Bạn có thể sử dụng biểu đồ đầu tiên để tìm ngưỡng cần sử dụng, nhưng cách này không chính xác lắm. Thay vào đó, bạn có thể tìm kiếm ngưỡng thấp nhất mang lại cho bạn độ chính xác ít nhất 90%. Để làm điều này, bạn có thể sử dụng phương thức

phương thức `argmax()` của mảng NumPy. Phương thức này trả về chỉ số đầu tiên của giá trị lớn nhất, trong trường hợp này có nghĩa là giá trị `True` đầu tiên:

```
>>> idx_for_90_precision = (precisions >= 0.90).argmax()
>>> threshold_for_90_precision = thresholds[idx_for_90_precision]
>>> threshold_for_90_precision
np.float64(3370.0194991439557)
```

Để thực hiện dự đoán (trên tập huấn luyện trước mắt), thay vì gọi phương thức `predict()` của mô hình phân loại, bạn có thể chạy đoạn mã sau:

```
y_train_pred_90 = (y_scores >= threshold_for_90_precision)
```

Hãy kiểm tra độ chính xác và độ thu hồi của các dự đoán này:

```
>>> precision_score(y_train_5, y_train_pred_90)
0.9000345901072293
>>> recall_at_90_precision = recall_score(y_train_5, y_train_pred_90)
>>> recall_at_90_precision
0.4799852425751706
```

Tuyệt vời, bạn đã có một mô hình phân loại với độ chính xác 90%! Như bạn có thể thấy, việc tạo ra một mô hình phân loại với gần như bất kỳ độ chính xác nào bạn muốn đều khá đơn giản: chỉ cần đặt ngưỡng đủ cao là xong. Nhưng khoan đã, đừng vội: một mô hình phân loại có độ chính xác cao sẽ không thực sự hữu ích nếu độ nhạy của nó quá thấp! Đối với nhiều ứng dụng, độ nhạy 48% sẽ không hề tốt chút nào.



Nếu ai đó nói: "Hãy đạt độ chính xác 99%", bạn nên hỏi lại: "Với độ thu hồi là bao nhiêu?"

Kể từ Scikit-Learn 1.5, có hai lớp mới mà bạn có thể sử dụng để điều chỉnh ngưỡng quyết định dễ dàng hơn:

- Lớp `FixedThresholdClassifier` cho phép bạn bao bọc một mô hình phân loại nhị phân và thiết lập ngưỡng mong muốn một cách thủ công. Nếu mô hình cơ sở có phương thức `predict_proba()`, thì ngưỡng phải là một giá trị nằm trong khoảng từ 0 đến 1 (giá trị mặc định là 0,5). Ngược lại, nó phải là một điểm số quyết định, tương đương với đầu ra của hàm `decision_function()` của mô hình (giá trị mặc định là 0).
- Lớp `TunedThresholdClassifierCV` sử dụng phương pháp kiểm chứng chéo k-fold để tự động tìm ngưỡng tối ưu cho một chỉ số cụ thể. Theo mặc định, nó cố gắng tìm ngưỡng tối đa hóa *độ chính xác cân bằng* của mô hình: đó là giá trị trung bình của độ nhạy (recall) của từng lớp. Tuy nhiên, bạn có thể chọn một chỉ số khác để tối ưu hóa (xem tài liệu để biết danh sách đầy đủ các tùy chọn).

Đường cong ROC

Đường cong *đặc tính hoạt động của bộ thu* (ROC) là một công cụ phổ biến khác được sử dụng với các bộ phân loại nhị phân. Nó rất giống với đường cong độ chính xác/độ thu hồi, nhưng thay vì vẽ đồ thị độ chính xác so với độ thu hồi, đường cong ROC vẽ đồ thị *tỷ lệ dương tính thật* (một tên gọi khác của độ thu hồi) so với *tỷ lệ dương tính giả* (FPR). FPR (còn gọi là *tỷ lệ sai*) là tỷ lệ các trường hợp âm tính bị phân loại sai thành dương tính. Nó bằng 1 – *tỷ lệ âm tính thực* (TNR), tức là tỷ lệ các trường hợp âm tính được phân loại chính xác là âm tính. TNR còn được gọi là *độ đặc hiệu*. Do đó, đường cong ROC vẽ *độ nhạy* (độ thu hồi) so với 1 – *độ đặc hiệu*.

Để vẽ đường cong ROC, trước tiên bạn sử dụng hàm `roc_curve()` để tính toán TPR và FPR cho các giá trị ngưỡng khác nhau:

```
from sklearn.metrics import roc_curve

fpr, tpr, thresholds = roc_curve(y_train_5, y_scores)
```

Sau đó, bạn có thể vẽ đồ thị FPR theo TPR bằng Matplotlib. Đoạn mã sau đây tạo ra đồ thị trong Hình 3-7. Để tìm điểm tương ứng với độ chính xác 90%, chúng ta cần xác định chỉ số của ngưỡng mong muốn. Vì các ngưỡng được liệt kê theo thứ tự giảm dần trong trường hợp này, chúng ta sử dụng `<=` thay vì `>=` ở dòng đầu tiên:

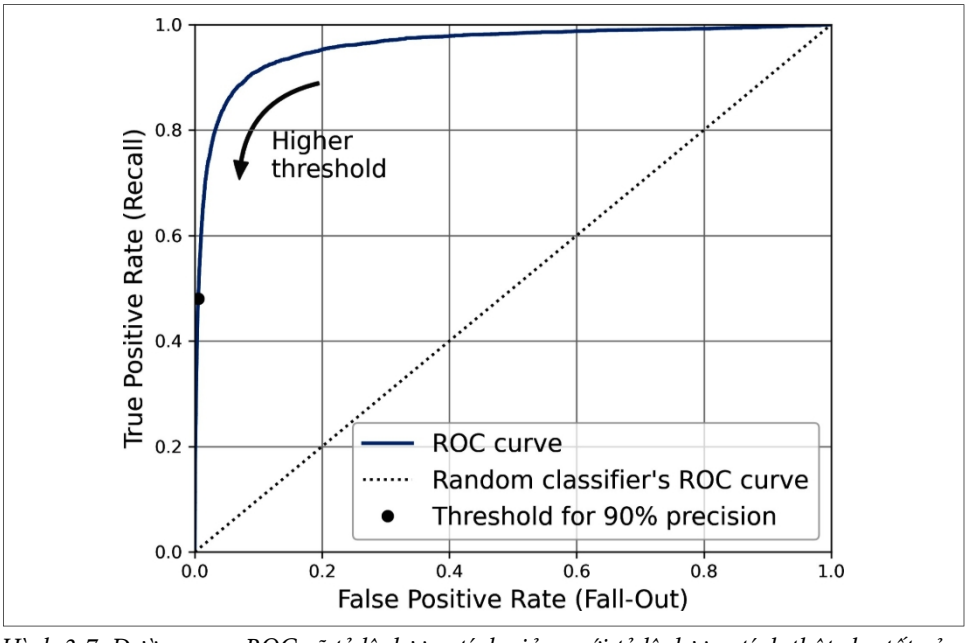
```
idx_for_threshold_at_90 = (thresholds <= threshold_for_90_precision).argmax() tpr_90,
fpr_90 = tpr[idx_for_threshold_at_90], fpr[idx_for_threshold_at_90]

plt.plot(fpr, tpr, linewidth=2, label="Đường cong ROC")
plt.plot([0, 1], [0, 1], 'k:', label="Đường cong ROC của bộ phân loại ngẫu
nhiên") plt.plot([fpr_90], [tpr_90], "ko", label="Ngưỡng cho độ chính xác 90%")
[...] # làm đẹp hình vẽ: thêm nhãn, lưới, chú thích, mũi tên và văn bản plt.show()
```

Một lần nữa, đây là một sự đánh đổi: độ nhạy (TPR) càng cao, mô hình phân loại càng tạo ra nhiều kết quả dương giả (FPR). Đường chấm chấm biểu thị đường cong ROC của một mô hình phân loại hoàn toàn ngẫu nhiên; một mô hình phân loại tốt sẽ nằm càng xa đường đó càng tốt (hướng về góc trên bên trái).

Một cách để so sánh các mô hình phân loại là đo *diện tích dưới đường cong* (AUC). Một mô hình phân loại hoàn hảo sẽ có AUC ROC bằng 1, trong khi một mô hình phân loại hoàn toàn ngẫu nhiên sẽ có AUC ROC bằng 0,5. Scikit-Learn cung cấp một hàm để ước tính AUC ROC:

```
>>> from sklearn.metrics import roc_auc_score
>>> roc_auc_score(y_train_5, y_scores)
np.float64(0.9604938554008616)
```



Hình 3-7. Đường cong ROC vẽ tỷ lệ dương tính giả so với tỷ lệ dương tính thật cho tất cả các ngưỡng có thể; vòng tròn màu đen đánh dấu tỷ lệ đã chọn (với độ chính xác 90% và độ thu hồi 48%)



Vì đường cong ROC rất giống với đường cong độ chính xác/độ nhạy (PR), bạn có thể thắc mắc làm thế nào để quyết định nên sử dụng đường cong nào. Theo nguyên tắc chung, bạn nên ưu tiên đường cong PR khi lớp dương hiếm gặp hoặc khi bạn quan tâm đến các trường hợp dương tính giả hơn là các trường hợp âm tính giả. Trong trường hợp khác, hãy sử dụng đường cong ROC. Ví dụ, khi xem xét đường cong ROC trước đó (và điểm AUC của ROC), bạn có thể nghĩ rằng mô hình phân loại này rất tốt. Tuy nhiên, điều này chủ yếu là do số lượng mẫu dương (5) rất ít so với mẫu âm (không phải 5). Ngược lại, đường cong PR cho thấy rõ ràng rằng mô hình phân loại này vẫn còn nhiều tiềm năng cải thiện: đường cong thực sự có thể nằm gần góc trên bên phải hơn (xem lại Hình 3-6).

Bây giờ, hãy tạo một `RandomForestClassifier`, để chúng ta có thể so sánh đường cong PR và điểm F_1 của nó với `SGDClassifier`:

```
from sklearn.ensemble import RandomForestClassifier

forest_clf = RandomForestClassifier(random_state=42)
```

Hàm `precision_recall_curve()` yêu cầu nhãn và điểm số cho từng bản ghi, vì vậy chúng ta cần huấn luyện mô hình phân loại rừng ngẫu nhiên và khiến nó gán một

điểm số cho từng trường hợp. Tuy nhiên, lớp `RandomForestClassifier` không có phương thức `decision_function()`, do cách thức hoạt động của nó (chúng ta sẽ đề cập đến điều này trong Chương 6). May mắn thay, nó có phương thức `predict_proba()` trả về xác suất ước tính của từng lớp cho mỗi bản ghi, và chúng ta có thể sử dụng xác suất của lớp dương làm điểm số, do đó hàm `precision_recall_curve()` sẽ hoạt động.[4] Chúng ta có thể gọi hàm `cross_val_predict()` để huấn luyện mô hình `RandomForestClassifier` bằng phương pháp kiểm tra chéo và khiến nó dự đoán xác suất các lớp cho mỗi hình ảnh như sau:

```
y_probas_forest = cross_val_predict(forest_clf, X_train, y_train_5, cv=3,
                                    method="predict_proba")
```

Hãy xem xét các xác suất lớp được ước tính cho hai hình ảnh đầu tiên trong tập huấn luyện:

```
>>> y_probas_forest[:2]
array([[0.11, 0.89],
       [0.99, 0.01]])
```

Mô hình dự đoán rằng hình ảnh đầu tiên là dương tính với xác suất 89%, và dự đoán rằng hình ảnh thứ hai là âm tính với xác suất 99%. Vì mỗi hình ảnh chỉ có thể là dương tính hoặc âm tính, nên tổng các xác suất ước tính trong mỗi hàng bằng 100%.



Đây là các xác suất *ước tính*, không phải xác suất thực tế. Ví dụ, nếu xem xét tất cả các hình ảnh mà mô hình phân loại là dương tính với xác suất ước tính nằm trong khoảng từ 50% đến 60%, thì khoảng 94% trong số đó thực sự là dương tính. Do đó, các xác suất ước tính của mô hình trong trường hợp này quá thấp—nhưng mô hình cũng có thể quá tự tin. Lớp `CalibratedClassifierCV` từ gói `sklearn.calibration` có thể hiệu chỉnh các xác suất ước tính bằng cách sử dụng kiểm tra chéo (cross-validation), giúp chúng gần với xác suất thực tế hơn nhiều (xem notebook để xem ví dụ mã nguồn). Điều này rất quan trọng trong một số tình huống, chẳng hạn như chẩn đoán y tế, đánh giá rủi ro tài chính hoặc phát hiện gian lận.

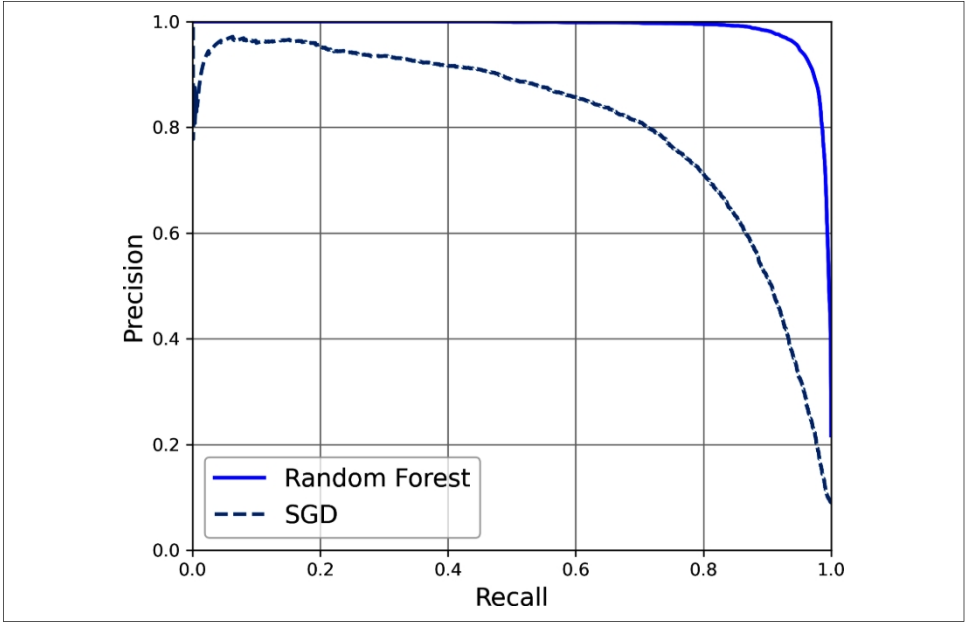
Cột thứ hai chứa các xác suất ước tính cho lớp dương tính, vì vậy hãy truyền chúng vào hàm `precision_recall_curve()`:

```
y_scores_forest = y_probas_forest[:, 1]
precisions_forest, recalls_forest, thresholds_forest = precision_recall_curve( y_train_5,
    y_scores_forest)
```

Bây giờ chúng ta đã sẵn sàng để vẽ đường cong PR. Việc vẽ đường cong PR đầu tiên cũng rất hữu ích để so sánh chúng (Hình 3-8):

4 Các bộ phân loại Scikit-Learn luôn có phương thức `decision_function()` hoặc `predict_proba()`, hoặc đôi khi là cả hai.

```
plt.plot(recalls_forest, precisions_forest, "b-", linewidth=2, label="Random
         Forest")
plt.plot(recalls, precisions, "--", linewidth=2, label="SGD") [...] #
làm đẹp hình vẽ: thêm nhãn, lưới và chú thích plt.show()
```



Hình 3-8. So sánh các đường cong PR: mô hình phân loại Random Forest vượt trội hơn mô hình SGD vì đường cong PR của nó nằm gần góc trên bên phải hơn nhiều và có giá trị AUC cao hơn

Như bạn có thể thấy trong Hình 3-8, đường cong PR của RandomForestClassifier trông tốt hơn nhiều so với của SGDClassifier: nó nằm gần góc trên bên phải hơn nhiều. Điểm F_1và điểm AUC ROC của nó cũng tốt hơn đáng kể:

```
>>> y_train_pred_forest = y_probas_forest[:, 1] >= 0.5 # xác suất dương ≥ 50%
>>> f1_score(y_train_5, y_train_pred_forest)
0.9274509803921569
>>> roc_auc_score(y_train_5, y_scores_forest)
0.9983436731328145
```

Hãy thử đo lường các chỉ số độ chính xác (precision) và độ nhạy (recall): bạn sẽ thấy độ chính xác khoảng 99,0% và độ nhạy khoảng 87,3%. Không tệ chút nào!

Bây giờ bạn đã biết cách huấn luyện các bộ phân loại nhị phân, chọn thước đo phù hợp cho tác vụ của mình, đánh giá các bộ phân loại bằng cách sử dụng xác nhận chéo, chọn sự cân bằng giữa độ chính xác và độ thu hồi phù hợp với nhu cầu của bạn, và sử dụng một số thước đo và đường cong để so sánh các mô hình khác nhau. Bạn đã sẵn sàng để thử phát hiện nhiều hơn chỉ các số 5.

Phân loại đa lớp

Trong khi các mô hình phân loại nhị phân phân biệt giữa hai lớp, *các mô hình phân loại đa lớp* (còn gọi là *mô hình phân loại đa thức*) có thể phân biệt giữa nhiều hơn hai lớp.

Một số mô hình phân loại trong Scikit-Learn (ví dụ: `LogisticRegression`, `RandomForestClassifier` và `GaussianNB`) có khả năng xử lý nhiều lớp một cách bản địa. Các mô hình khác là mô hình phân loại nhị phân thuần túy (ví dụ: `SGDClassifier` và `SVC`). Tuy nhiên, có nhiều chiến lược khác nhau mà bạn có thể sử dụng để thực hiện phân loại đa lớp bằng cách kết hợp nhiều mô hình phân loại nhị phân.

Một cách để xây dựng hệ thống có thể phân loại hình ảnh các chữ số thành 10 lớp (từ 0 đến 9) là huấn luyện 10 bộ phân loại nhị phân, mỗi bộ dành cho một chữ số (bộ phát hiện số 0, bộ phát hiện số 1, bộ phát hiện số 2, v.v.). Sau đó, khi bạn muốn phân loại một hình ảnh, bạn lấy điểm quyết định từ mỗi bộ phân loại cho hình ảnh đó và chọn lớp có bộ phân loại cho ra điểm cao nhất. Đây được gọi là chiến lược *"một so với phần còn lại"* (OvR), hoặc đôi khi là *"một so với tất cả"* (OvA).

Một chiến lược khác là huấn luyện một bộ phân loại nhị phân cho mỗi cặp chữ số: một bộ để phân biệt 0 và 1, một bộ khác để phân biệt 0 và 2, một bộ khác cho 1 và 2, v.v. Điều này được gọi là chiến lược *"một chống một"* (OvO). Nếu có N lớp, bạn cần huấn luyện $N \times (N - 1) / 2$ bộ phân loại. Đối với bài toán MNIST, điều này có nghĩa là phải huấn luyện tới 45 mô hình phân loại nhị phân! Khi muốn phân loại một hình ảnh, bạn phải đưa hình ảnh đó qua tất cả 45 mô hình phân loại và xem lớp nào thắng nhiều "cuộc đối đầu" nhất. Ưu điểm chính của OvO là mỗi mô hình phân loại chỉ cần được huấn luyện trên phần tập huấn luyện chứa hai lớp mà nó phải phân biệt.

Một số thuật toán (chẳng hạn như các mô hình phân loại máy vectơ hỗ trợ) có hiệu suất giảm đáng kể khi tập huấn luyện lớn lên. Đối với các thuật toán này, OvO được ưu tiên vì việc huấn luyện nhiều mô hình trên các tập huấn luyện nhỏ sẽ nhanh hơn so với việc huấn luyện ít mô hình trên các tập huấn luyện lớn. Tuy nhiên, đối với hầu hết các thuật toán phân loại nhị phân, OvR lại được ưu tiên hơn.

Scikit-Learn sẽ phát hiện khi bạn cố gắng sử dụng một thuật toán phân loại nhị phân cho tác vụ phân loại đa lớp, và nó sẽ tự động chạy OvR hoặc OvO, tùy thuộc vào thuật toán. Hãy thử điều này với một mô hình phân loại máy vectơ hỗ trợ (SVM) sử dụng lớp `sklearn.svm.SVC` (xem chương trực tuyến về SVM tại *https://homl.info*). Chúng ta sẽ chỉ huấn luyện trên 2.000 hình ảnh đầu tiên, nếu không thì sẽ mất rất nhiều thời gian:

```
from sklearn.svm import SVC

svm_clf = SVC(random_state=42)
svm_clf.fit(X_train[:2000], y_train[:2000]) # y_train, không phải y_train_5
```

Thật dễ dàng! Chúng ta đã huấn luyện mô hình `SVC` bằng cách sử dụng các lớp mục tiêu gốc từ 0 đến 9 (`y_train`), thay vì các lớp mục tiêu "5 so với phần còn lại" (`y_train_5`). Vì có 10 lớp (tức là nhiều hơn 2), Scikit-Learn đã sử dụng chiến lược OvO và huấn luyện 45 bộ phân loại nhị phân. Bây giờ hãy thực hiện dự đoán trên một hình ảnh:

```
>>> svm_clf.predict([some_digit])
array(['5'], dtype=object)
```

Đúng rồi! Đoạn mã này thực tế đã thực hiện 45 dự đoán—mỗi cặp lớp một lần—và chọn ra lớp thắng nhiều trận đối đầu nhất.[5] Nếu bạn gọi phương thức `decision_function()`, bạn sẽ thấy nó trả về 10 điểm số cho mỗi bản ghi: một điểm cho mỗi lớp. Mỗi lớp nhận được điểm số bằng số trận đối đầu thắng cộng hoặc trừ một điều chỉnh nhỏ (max ±0.33) để phân định các trường hợp hòa, dựa trên điểm số của bộ phân loại:

```
>>> some_digit_scores = svm_clf.decision_function([some_digit])
>>> some_digit_scores.round(2)
array([[ 3.79, 0.73, 6.06, 8.3 , -0.29, 9.3 , 1.75, 2.77, 7.21,
         4.82]])
```

Điểm số cao nhất là 9.3, và đó thực sự là điểm số tương ứng với lớp 5:

```
>>> class_id = some_digit_scores.argmax()
>>> class_id
np.int64(5)
```

Khi một mô hình phân loại được huấn luyện, nó lưu trữ danh sách các lớp mục tiêu trong thuộc tính `` `classes_` ``, được sắp xếp theo giá trị. Trong trường hợp của MNIST, chỉ số của mỗi lớp trong mảng `` `classes_` `` trùng khớp thuận tiện với chính lớp đó (ví dụ: lớp ở chỉ số 5 chính là lớp `'5'`), nhưng nói chung bạn sẽ không may mắn như vậy; bạn sẽ cần tra cứu nhãn lớp như sau:

```
>>> svm_clf.classes_
array(['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'], dtype=object)
>>> svm_clf.classes_[class_id] '5'
```

Nếu bạn muốn buộc Scikit-Learn sử dụng phương pháp one-versus-one hoặc one-versus-the-rest, bạn có thể sử dụng các lớp `OneVsOneClassifier` hoặc `OneVsRestClassifier`. Chỉ cần tạo một đối tượng và truyền một bộ phân loại vào hàm tạo của nó (nó thậm chí không nhất thiết phải là bộ phân loại nhị phân). Ví dụ, đoạn mã này tạo ra một bộ phân loại đa lớp sử dụng chiến lược OvR, dựa trên `SVC`:

```
from sklearn.multiclass import OneVsRestClassifier

ovr_clf = OneVsRestClassifier(SVC(random_state=42))
ovr_clf.fit(X_train[:2000], y_train[:2000])
```

Hãy thực hiện dự đoán và kiểm tra số lượng mô hình phân loại đã được huấn luyện:

```
>>> ovr_clf.predict([some_digit])
array(['5'], dtype='<U1')
>>> len(ovr_clf.estimators_) 10
```

5 Trong trường hợp có kết quả hòa, lớp đầu tiên sẽ được chọn, trừ khi bạn đặt tham số `break_ties` thành `True`, trong trường hợp đó, kết quả hòa sẽ được phân định bằng cách sử dụng kết quả của hàm `decision_function()`.

Việc huấn luyện `SGDClassifier` trên tập dữ liệu đa lớp và sử dụng nó để đưa ra dự đoán cũng dễ dàng không kém:

```
>>> sgd_clf = SGDClassifier(random_state=42)
>>> sgd_clf.fit(X_train, y_train)
>>> sgd_clf.predict([some_digit])
array(['3'], dtype='<U1')
```

Ồ, kết quả này không chính xác. Lỗi dự đoán vẫn có thể xảy ra! Lần này, Scikit-Learn đã sử dụng chiến lược OvR ở chế độ nền: vì có 10 lớp, nên nó đã huấn luyện 10 bộ phân loại nhị phân. Phương thức `decision_function()` hiện trả về một giá trị cho mỗi lớp. Hãy xem điểm số mà bộ phân loại SGD gán cho từng lớp:

```
>>> sgd_clf.decision_function([some_digit]).round()
array([[-31893., -34420., -9531.,      1824., -22320., -1386., -26189.,
        -16148., -4604., -12051.]])
```

Bạn có thể thấy rằng mô hình phân loại không thực sự tự tin vào dự đoán của mình: hầu hết các điểm số đều rất âm, trong khi lớp 3 có điểm số là +1.824, và lớp 5 cũng không kém xa lắm với –1.386. Tất nhiên, bạn sẽ muốn đánh giá mô hình phân loại này trên nhiều hình ảnh hơn. Vì số lượng hình ảnh trong mỗi lớp gần như bằng nhau, nên chỉ số độ chính xác là phù hợp. Như thường lệ, bạn có thể sử dụng hàm `cross_val_score()` để đánh giá mô hình:

```
>>> cross_val_score(sgd_clf, X_train, y_train, cv=3, scoring="accuracy") array([0.87365,
0.85835, 0.8689 ])
```

Kết quả đạt trên 85,8% trên tất cả các tập dữ liệu kiểm tra. Nếu bạn sử dụng một mô hình phân loại ngẫu nhiên, độ chính xác sẽ chỉ đạt 10%, vì vậy đây không phải là kết quả quá tệ, nhưng bạn vẫn có thể cải thiện hơn nữa. Chỉ cần chuẩn hóa dữ liệu đầu vào (như đã thảo luận trong Chương 2) là có thể nâng độ chính xác lên trên 89,1%:

```
>>> from sklearn.preprocessing import StandardScaler
>>> scaler = StandardScaler()
>>> X_train_scaled = scaler.fit_transform(X_train.astype("float64"))
>>> cross_val_score(sgd_clf, X_train_scaled, y_train, cv=3, scoring="accuracy")
array([0.8983, 0.891 , 0.9018])
```

Phân tích lỗi

Nếu đây là một dự án thực tế, bạn sẽ thực hiện các bước trong danh sách kiểm tra dự án học máy của mình (xem *https://homl.info/checklist*). Bạn sẽ khám phá các tùy chọn chuẩn bị dữ liệu, thử nghiệm nhiều mô hình, chọn lọc những mô hình tốt nhất, tinh chỉnh các tham số siêu (hyperparameters) của chúng bằng `GridSearchCV` và tự động hóa càng nhiều càng tốt. Ở đây, chúng ta sẽ giả định rằng bạn đã tìm được một mô hình đầy hứa hẹn và muốn tìm cách cải thiện nó. Một cách để làm điều này là phân tích các loại lỗi mà mô hình mắc phải.

Đầu tiên, hãy xem ma trận nhầm lẫn. Để làm điều này, trước tiên bạn cần thực hiện dự đoán bằng hàm `cross_val_predict()`; sau đó, bạn có thể truyền các nhãn và dự đoán vào hàm `confusion_matrix()`, giống như bạn đã làm trước đó. Tuy nhiên, vì hiện tại có 10 lớp thay vì 2, ma trận nhầm lẫn sẽ chứa khá nhiều số và có thể khó đọc.

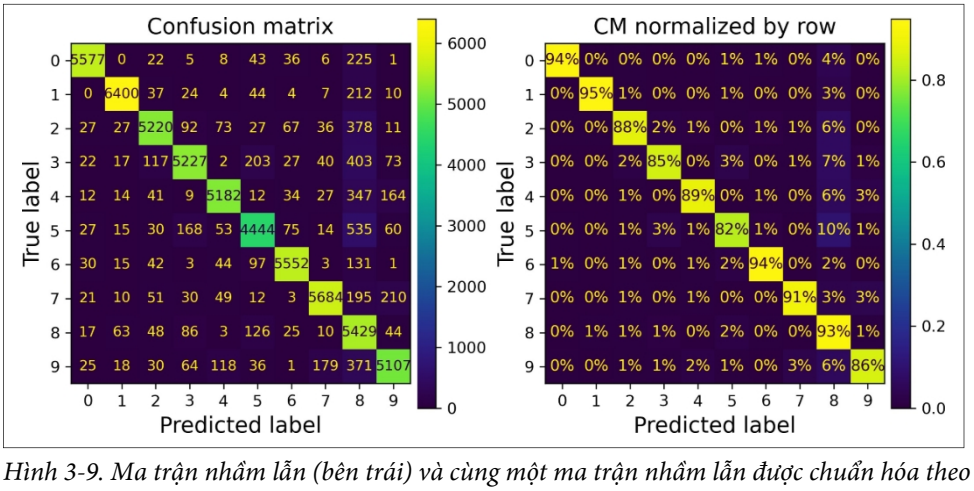
Một biểu đồ màu của ma trận nhầm lẫn sẽ dễ phân tích hơn nhiều. Để vẽ biểu đồ như vậy, hãy sử dụng hàm `ConfusionMatrixDisplay.from_predictions()` như sau:

```
from sklearn.metrics import ConfusionMatrixDisplay

y_train_pred = cross_val_predict(sgd_clf, X_train_scaled, y_train, cv=3)
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred) plt.show()
```

Kết quả thu được là biểu đồ bên trái trong Hình 3-9. Ma trận nhầm lẫn này trông khá tốt: hầu hết các hình ảnh nằm trên đường chéo chính, điều này có nghĩa là chúng đã được phân loại chính xác. Lưu ý rằng ô trên đường chéo ở hàng #5 và cột #5 trông hơi tối hơn so với các chữ số khác. Điều này có thể là do mô hình mắc nhiều lỗi hơn với chữ số 5, hoặc do số lượng chữ số 5 trong tập dữ liệu ít hơn so với các chữ số khác. Đó là lý do tại sao việc chuẩn hóa ma trận nhầm lẫn bằng cách chia mỗi giá trị cho tổng số hình ảnh trong lớp (thực) tương ứng (tức là chia cho tổng của hàng đó) là rất quan trọng. Việc này có thể thực hiện đơn giản bằng cách đặt `` `normalize="true"` ``. Chúng ta cũng có thể chỉ định tham số `` `values_format=".0%"` `` để hiển thị tỷ lệ phần trăm không có số thập phân. Đoạn mã sau đây tạo ra biểu đồ ở bên phải trong Hình 3-9:

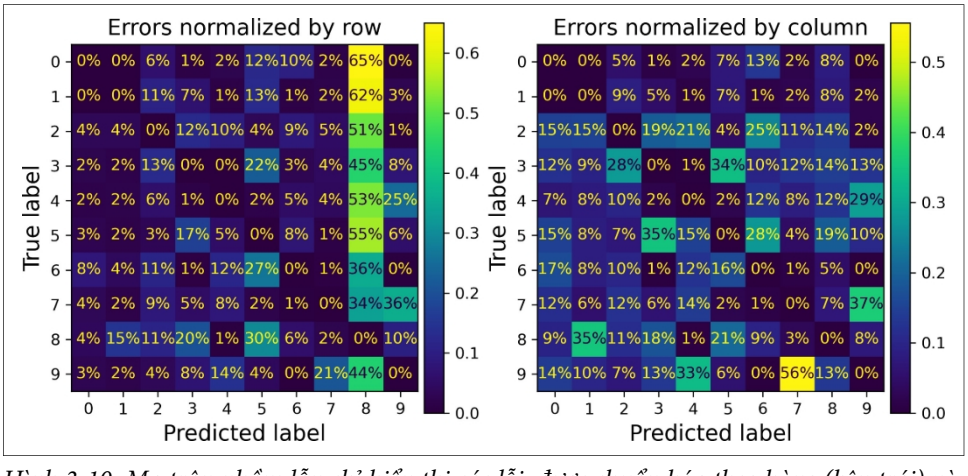
```
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred,
                                        normalize="true", values_format=".0%")
plt.show()
```



Hình 3-9. Ma trận nhầm lẫn (bên trái) và cùng một ma trận nhầm lẫn được chuẩn hóa theo hàng (bên phải)

Giờ đây, chúng ta có thể dễ dàng thấy rằng chỉ 82% hình ảnh số 5 được phân loại chính xác. Lỗi phổ biến nhất mà mô hình mắc phải với hình ảnh số 5 là phân loại nhầm chúng thành số 8: điều này xảy ra với 10% tổng số hình ảnh số 5. Tuy nhiên, chỉ 2% hình ảnh số 8 bị phân loại nhầm thành số 5; ma trận nhầm lẫn thường không đối xứng! Nếu quan sát kỹ, bạn sẽ nhận thấy rằng nhiều chữ số đã bị phân loại nhầm thành 8s, nhưng điều này không dễ dàng nhận ra ngay từ biểu đồ này. Nếu muốn làm nổi bật các lỗi hơn, bạn có thể thử đặt trọng số bằng 0 cho các dự đoán chính xác. Đoạn mã sau đây thực hiện chính xác điều đó và tạo ra biểu đồ bên trái trong Hình 3-10:

```
sample_weight = (y_train_pred != y_train) ConfusionMatrixDisplay.from_predictions(y_train,
y_train_pred,
                                                  sample_weight=sample_weight,
                                                  normalize="true", values_format=".0%")
plt.show()
```



Hình 3-10. Ma trận nhầm lẫn chỉ hiển thị các lỗi, được chuẩn hóa theo hàng (bên trái) và theo cột (bên phải)

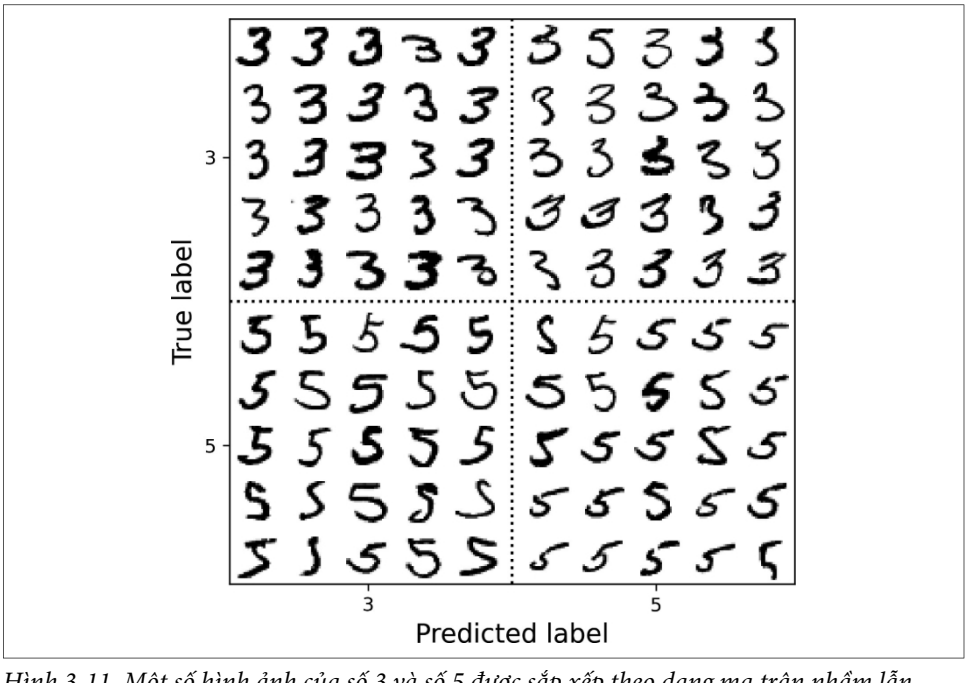
Giờ đây, bạn có thể thấy rõ hơn nhiều các loại lỗi mà mô hình phân loại mắc phải. Cột tương ứng với lớp 8 hiện rất nổi bật, điều này xác nhận rằng nhiều hình ảnh đã bị phân loại sai thành lớp 8. Trên thực tế, đây là lỗi phân loại phổ biến nhất đối với hầu hết các lớp. Tuy nhiên, hãy cẩn thận khi giải thích các tỷ lệ phần trăm trong biểu đồ này: hãy nhớ rằng chúng ta đã loại trừ các dự đoán chính xác. Ví dụ, con số 36% ở hàng số 7, cột số 9 trong lưới bên trái *không* có nghĩa là 36% tổng số hình ảnh thuộc lớp 7 bị phân loại nhầm thành lớp 9. Điều đó có nghĩa là 36% số *lỗi* mà mô hình mắc phải trên các hình ảnh thuộc lớp 7 là các trường hợp phân loại nhầm thành lớp 9. Trên thực tế, chỉ có 3% hình ảnh số 7 bị phân loại nhầm thành số 9, như bạn có thể thấy trong biểu đồ bên phải ở Hình 3-9.

Bạn cũng có thể chuẩn hóa ma trận nhầm lẫn theo cột thay vì theo hàng: nếu bạn đặt `normalize="pred"`, bạn sẽ nhận được biểu đồ bên phải trong Hình 3-10. Ví dụ, bạn có thể thấy rằng 56% các trường hợp số 7 bị phân loại sai thực chất là số 9.

Phân tích ma trận nhầm lẫn thường giúp bạn có được những hiểu biết về cách cải thiện mô hình phân loại. Nhìn vào các biểu đồ này, có vẻ như bạn nên tập trung nỗ lực vào việc giảm số trường hợp nhầm lẫn thành số 8. Ví dụ, bạn có thể thử thu thập thêm dữ liệu huấn luyện cho các chữ số trông giống số 8 (nhưng thực tế không phải) để mô hình phân loại có thể học cách phân biệt chúng với số 8 thực sự. Hoặc bạn có thể thiết kế các đặc trưng mới để hỗ trợ mô hình phân loại — ví dụ, viết thuật toán đếm số vòng lặp đóng (ví dụ: số 8 có hai vòng lặp, số 6 có một vòng lặp, số 5 không có vòng lặp nào). Hoặc bạn có thể tiền xử lý hình ảnh (ví dụ: sử dụng Scikit-Image, Pillow hoặc OpenCV) để làm nổi bật hơn các mẫu như vòng lặp đóng.

Phân tích từng lỗi cụ thể cũng là một cách tốt để hiểu rõ hơn về cách thức hoạt động của mô hình phân loại và lý do tại sao nó lại thất bại. Ví dụ, hãy vẽ các mẫu số 3 và số 5 theo kiểu ma trận nhầm lẫn (Hình 3-11):

```
cl_a, cl_b = '3', '5'
X_aa = X_train[(y_train == cl_a) & (y_train_pred == cl_a)]
X_ab = X_train[(y_train == cl_a) & (y_train_pred == cl_b)]
X_ba = X_train[(y_train == cl_b) & (y_train_pred == cl_a)]
X_bb = X_train[(y_train == cl_b) & (y_train_pred == cl_b)]
[...] # vẽ tất cả các hình ảnh trong X_aa, X_ab, X_ba, X_bb theo kiểu ma trận nhầm lẫn
```



Hình 3-11. Một số hình ảnh của số 3 và số 5 được sắp xếp theo dạng ma trận nhầm lẫn

Như bạn có thể thấy, một số chữ số mà bộ phân loại nhận diện sai (tức là ở các khối dưới cùng bên trái và trên cùng bên phải) được viết rất cẩu thả đến mức ngay cả con người cũng khó phân loại chúng. Tuy nhiên, hầu hết các hình ảnh bị phân loại sai dường như là những lỗi rõ ràng đối với chúng ta.

Có thể khó hiểu tại sao mô hình phân loại lại mắc những sai lầm đó, nhưng hãy nhớ rằng bộ não con người là một hệ thống nhận diện mẫu tuyệt vời, và hệ thống thị giác của chúng ta thực hiện rất nhiều quá trình tiền xử lý phức tạp trước khi bất kỳ thông tin nào thậm chí đến được ý thức của chúng ta. Vì vậy, việc nhiệm vụ này có vẻ đơn giản không có nghĩa là nó thực sự đơn giản. Hãy nhớ lại rằng chúng ta đã sử dụng một `SGDClassifier` đơn giản, vốn chỉ là một mô hình tuyến tính: tất cả những gì nó làm là gán một trọng số cho mỗi lớp cho từng pixel, và khi gặp một hình ảnh mới, nó chỉ cộng tổng các cường độ pixel đã được gán trọng số để tính điểm cho từng lớp. Vì số 3 và số 5 chỉ khác nhau bởi vài pixel, mô hình này sẽ dễ dàng nhầm lẫn giữa chúng.

Sự khác biệt chính giữa số 3 và số 5 nằm ở vị trí của đường thẳng nhỏ nối đường trên với cung dưới. Nếu bạn vẽ số 3 với điểm nối bị dịch chuyển nhẹ sang trái, bộ phân loại có thể phân loại nó thành số 5, và ngược lại. Nói cách khác, bộ phân loại này khá nhạy cảm với sự dịch chuyển và xoay của hình ảnh. Một cách để giảm nhầm lẫn giữa số 3 và số 5 là tiền xử lý hình ảnh để đảm bảo chúng được căn giữa tốt và không bị xoay quá nhiều. Tuy nhiên, điều này có thể không dễ dàng vì nó đòi hỏi phải dự đoán chính xác góc xoay của từng hình ảnh. Một cách tiếp cận đơn giản hơn nhiều là mở rộng tập huấn luyện bằng các biến thể của hình ảnh huấn luyện bị dịch chuyển và xoay nhẹ. Điều này sẽ buộc mô hình phải học cách chịu đựng tốt hơn những biến thể như vậy. Đây được gọi là *mở rộng dữ liệu* (chúng ta sẽ đề cập đến vấn đề này trong Chương 12; xem thêm bài tập 2 ở cuối chương này).

Phân loại đa nhãn

Cho đến nay, mỗi mẫu luôn được gán vào chỉ một lớp. Tuy nhiên, trong một số trường hợp, bạn có thể muốn mô hình phân loại đưa ra nhiều lớp cho mỗi mẫu. Hãy xem xét một mô hình nhận diện khuôn mặt: nó nên làm gì nếu nhận diện được nhiều người trong cùng một bức ảnh? Nó nên gán một nhãn cho mỗi người mà nó nhận diện được. Giả sử mô hình phân loại đã được huấn luyện để nhận diện ba khuôn mặt: Alice, Bob và Charlie. Khi mô hình được hiển thị một bức ảnh có cả Alice và Charlie, nó nên trả về `[True, False, True]` (nghĩa là "Alice có, Bob không, Charlie có"). Hệ thống phân loại như vậy, trả về nhiều nhãn nhị phân, được gọi là hệ thống *phân loại đa nhãn*.

Chúng ta sẽ chưa đi sâu vào nhận diện khuôn mặt ngay lúc này, nhưng hãy xem xét một ví dụ đơn giản hơn, chỉ để minh họa:

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier

y_train_large = (y_train >= '7')
y_train_odd = (y_train.astype('int8') % 2 == 1)
y_multilabel = np.c_[y_train_large, y_train_odd]

knn_clf = KNeighborsClassifier()
knn_clf.fit(X_train, y_multilabel)
```

Đoạn mã này tạo ra một mảng `y_multilabel` chứa hai nhãn mục tiêu cho mỗi hình ảnh chữ số: nhãn đầu tiên cho biết chữ số đó có phải là chữ số lớn (7, 8 hoặc 9) hay không, và nhãn thứ hai

cho biết chữ số đó có là số lẻ hay không. Sau đó, đoạn mã tạo ra một đối tượng `KNeighborsClassifier`, hỗ trợ phân loại đa nhãn (không phải tất cả các bộ phân loại đều hỗ trợ), và huấn luyện mô hình này bằng mảng các nhãn mục tiêu. Bây giờ bạn có thể thực hiện dự đoán và lưu ý rằng nó trả về hai nhãn:

```
>>> knn_clf.predict([some_digit])
array([[False, True]])
```

Và kết quả hoàn toàn chính xác! Chữ số 5 thực sự không phải là số lớn (`False`) và là số lẻ (`True`).

Có nhiều cách để đánh giá một mô hình phân loại đa nhãn, và việc lựa chọn chỉ số phù hợp thực sự phụ thuộc vào dự án của bạn. Một cách tiếp cận là đo điểm F_1 cho từng nhãn riêng lẻ (hoặc bất kỳ chỉ số nào khác dành cho mô hình phân loại nhị phân đã được đề cập trước đó), sau đó đơn giản là tính điểm trung bình. Đoạn mã sau đây tính điểm F_1 trung bình trên tất cả các nhãn:

```
>>> y_train_knn_pred = cross_val_predict(knn_clf, X_train, y_multilabel, cv=3)
>>> f1_score(y_multilabel, y_train_knn_pred, average="macro") 0.976410265560605
```

Cách tiếp cận này giả định rằng tất cả các nhãn đều có mức độ quan trọng như nhau, điều này có thể không đúng trong thực tế. Đặc biệt, nếu bạn có nhiều hình ảnh của Alice hơn so với Bob hoặc Charlie, bạn có thể muốn gán trọng số cao hơn cho điểm số của mô hình phân loại trên các hình ảnh của Alice. Một lựa chọn đơn giản là gán cho mỗi nhãn một trọng số bằng với *độ hỗ trợ* của nó (tức là số lượng mẫu có nhãn mục tiêu đó). Để thực hiện điều này, chỉ cần đặt `average="weighted"` khi gọi hàm `f1_score()`.[6]

Nếu bạn muốn sử dụng một mô hình phân loại không hỗ trợ phân loại đa nhãn theo mặc định, chẳng hạn như `SVC`, một chiến lược khả thi là huấn luyện một mô hình cho mỗi nhãn. Tuy nhiên, chiến lược này có thể gặp khó khăn trong việc nắm bắt các mối quan hệ phụ thuộc giữa các nhãn. Ví dụ, một chữ số lớn (7, 8 hoặc 9) có khả năng là số lẻ cao gấp đôi so với số chẵn, nhưng mô hình phân loại cho nhãn "lẻ" không biết kết quả dự đoán của mô hình phân loại cho nhãn "lớn". Để giải quyết vấn đề này, các mô hình có thể được tổ chức thành một chuỗi: khi một mô hình đưa ra dự đoán, nó sẽ sử dụng các đặc trưng đầu vào cùng với tất cả các dự đoán từ các mô hình nằm trước nó trong chuỗi.

Tin vui là Scikit-Learn có một lớp gọi là `ClassifierChain` thực hiện chính xác điều đó! Theo mặc định, nó sẽ sử dụng các nhãn thực để huấn luyện, cung cấp cho mỗi mô hình các nhãn phù hợp tùy thuộc vào vị trí của chúng trong chuỗi. Tuy nhiên, nếu bạn thiết lập siêu tham số `` `cv` ``, nó sẽ sử dụng phương pháp kiểm chứng chéo (cross-validation) để thu được các dự đoán "sạch" (ngoài mẫu) từ mỗi mô hình đã được huấn luyện cho từng bản ghi trong tập huấn luyện, và các dự đoán này sau đó sẽ được sử dụng để huấn luyện tất cả các mô hình tiếp theo trong chuỗi. Lưu ý rằng thứ tự của các bộ phân loại trong chuỗi có thể ảnh hưởng đến hiệu suất cuối cùng. Dưới đây là một ví dụ minh họa

6 Scikit-Learn cung cấp một số tùy chọn tính trung bình khác và các chỉ số đánh giá cho mô hình phân loại đa nhãn; xem tài liệu tham khảo để biết thêm chi tiết.

cách tạo và huấn luyện `ClassifierChain` bằng chiến lược kiểm tra chéo. Như trước đó, chúng ta sẽ chỉ sử dụng 2.000 hình ảnh đầu tiên trong tập huấn luyện để tăng tốc độ:

```
from sklearn.multioutput import ClassifierChain

chain_clf = ClassifierChain(SVC(), cv=3, random_state=42) chain_clf.fit(X_train[:2000],
y_multilabel[:2000])
```

Bây giờ chúng ta có thể sử dụng `ClassifierChain` này để thực hiện dự đoán:

```
>>> chain_clf.predict([some_digit])
array([[0., 1.]])
```

Phân loại đa đầu ra

Loại tác vụ phân loại cuối cùng mà chúng ta sẽ thảo luận ở đây được gọi là *phân loại đa đầu ra – đa lớp* (hoặc đơn giản là *phân loại đa đầu ra*). Đây là sự tổng quát hóa của phân loại đa nhãn, trong đó mỗi nhãn có thể là đa lớp (tức là có thể có nhiều hơn hai giá trị khả dĩ).

Để minh họa điều này, hãy xây dựng một hệ thống loại bỏ nhiễu khỏi hình ảnh. Hệ thống này sẽ nhận đầu vào là một hình ảnh chữ số bị nhiễu và (hy vọng) sẽ đưa ra đầu ra là một hình ảnh chữ số sạch, được biểu diễn dưới dạng một mảng cường độ pixel, giống như các hình ảnh MNIST. Lưu ý rằng đầu ra của bộ phân loại là đa nhãn (một nhãn cho mỗi pixel) và mỗi nhãn có thể có nhiều giá trị (độ sáng pixel dao động từ 0 đến 255). Do đó, đây là một ví dụ về hệ thống phân loại đa đầu ra.

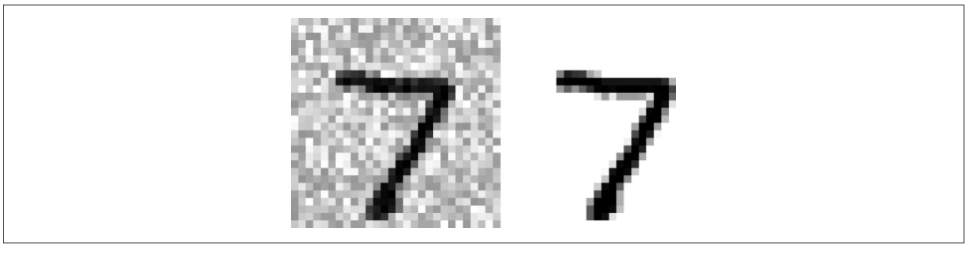


Ranh giới giữa phân loại và hồi quy đôi khi không rõ ràng, như trong ví dụ này. Có thể lập luận rằng việc dự đoán cường độ pixel gần với hồi quy hơn là phân loại. Hơn nữa, các hệ thống đa đầu ra không chỉ giới hạn ở các tác vụ phân loại; bạn thậm chí có thể xây dựng một hệ thống đưa ra nhiều nhãn cho mỗi thực thể, bao gồm cả nhãn lớp và nhãn giá trị.

Hãy bắt đầu bằng cách tạo tập huấn luyện và tập kiểm tra bằng cách lấy các hình ảnh MNIST và thêm nhiễu vào cường độ pixel của chúng, sử dụng phương thức `integers()` của bộ tạo số ngẫu nhiên. Các hình ảnh mục tiêu sẽ là các hình ảnh gốc:

```
rng = np.random.default_rng(seed=42)
noise_train = rng.integers(0, 100, (len(X_train), 784))
X_train_mod = X_train + noise_train
noise_test = rng.integers(0, 100, (len(X_test), 784))
X_test_mod = X_test + noise_test
y_train_mod = X_train
y_test_mod = X_test
```

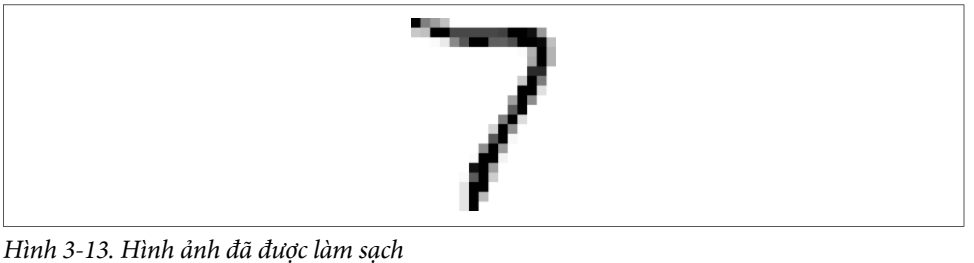
Hãy cùng xem qua hình ảnh đầu tiên trong tập dữ liệu kiểm tra (Hình 3-12). Đúng vậy, chúng ta đang xem trộm dữ liệu kiểm tra, nên chắc hẳn bạn đang cau mày lúc này.



Hình 3-12. Một hình ảnh nhiễu (bên trái) và hình ảnh mục tiêu sạch (bên phải)

Bên trái là hình ảnh đầu vào có nhiễu, còn bên phải là hình ảnh mục tiêu sạch. Bây giờ, hãy huấn luyện bộ phân loại và khiến nó làm sạch hình ảnh này (Hình 3-13):

```
knn_clf = KNeighborsClassifier()
knn_clf.fit(X_train_mod, y_train_mod) clean_digit
= knn_clf.predict([X_test_mod[0]])
plot_digit(clean_digit)
plt.show()
```



Hình 3-13. Hình ảnh đã được làm sạch

Trông khá giống với hình ảnh mục tiêu! Đây là phần kết thúc của bài hướng dẫn về phân loại. Giờ đây, bạn đã biết cách chọn các chỉ số phù hợp cho các tác vụ phân loại, lựa chọn sự cân bằng giữa độ chính xác và độ nhạy, so sánh các mô hình phân loại, và nói chung là xây dựng các hệ thống phân loại hiệu quả cho nhiều tác vụ khác nhau. Trong các chương tiếp theo, bạn sẽ tìm hiểu cách thức hoạt động thực sự của tất cả các mô hình học máy mà bạn đã sử dụng.

Bài tập

1. Hãy thử xây dựng một mô hình phân loại cho tập dữ liệu MNIST đạt độ chính xác trên 97% trên tập kiểm tra. Gợi ý: Mô hình `KNeighborsClassifier` hoạt động khá tốt cho tác vụ này; bạn chỉ cần tìm các giá trị tham số siêu phù hợp (hãy thử tìm kiếm theo lưới cho các tham số `trọng số` và `n_neighbors`).

2. Viết một hàm có thể dịch chuyển một hình ảnh MNIST theo bất kỳ hướng nào (trái, phải, lên hoặc xuống) một pixel.[7] Sau đó, đối với mỗi hình ảnh trong tập huấn luyện, tạo bốn bản sao đã dịch chuyển (mỗi hướng một bản) và thêm chúng vào tập huấn luyện. Cuối cùng, huấn luyện mô hình tốt nhất của bạn trên tập huấn luyện mở rộng này và đo độ chính xác của nó trên tập kiểm tra. Bạn sẽ thấy rằng mô hình của bạn giờ đây hoạt động còn tốt hơn nữa! Kỹ thuật mở rộng tập huấn luyện một cách nhân tạo này được gọi là *tăng cường dữ liệu* hoặc *mở rộng tập huấn luyện*.
3. Hãy thử xử lý tập dữ liệu Titanic. Kaggle là một nơi tuyệt vời để bắt đầu. Ngoài ra, bạn có thể tải dữ liệu từ *https://homl.info/titanic.tgz* và giải nén tệp tarball này giống như cách bạn đã làm với dữ liệu nhà ở trong Chương 2. Thao tác này sẽ cho bạn hai tệp CSV, *train.csv* và *test.csv*, mà bạn có thể tải bằng hàm `pandas.read_csv()`. Mục tiêu là huấn luyện một bộ phân loại có thể dự đoán cột `Survived` dựa trên các cột khác.
4. Xây dựng mô hình phân loại spam (một bài tập khó hơn):
   a. Tải xuống các ví dụ về thư rác và thư hợp lệ từ bộ dữ liệu công khai của Apache SpamAssassin.
   b. Giải nén các bộ dữ liệu và làm quen với định dạng dữ liệu.
   c. Chia dữ liệu thành tập huấn luyện và tập kiểm tra.
   d. Viết một quy trình chuẩn bị dữ liệu để chuyển đổi mỗi email thành một vectơ đặc trưng. Quy trình chuẩn bị của bạn nên biến một email thành một vectơ (thưa thớt) chỉ ra sự hiện diện hoặc vắng mặt của từng từ có thể có. Ví dụ, nếu tất cả email chỉ chứa bốn từ "Hello", "how", "are", "you", thì email "Hello you Hello Hello you" sẽ được chuyển đổi thành vectơ [1, 0, 0, 1] (nghĩa là ["Hello" có mặt, "how" vắng mặt, "are" vắng mặt, "you" có mặt]), hoặc [3, 0, 0, 2] nếu bạn muốn đếm số lần xuất hiện của từng từ.

      Bạn có thể muốn thêm các siêu tham số vào quy trình chuẩn bị dữ liệu để kiểm soát việc loại bỏ tiêu đề email, chuyển đổi mỗi email sang chữ thường, loại bỏ dấu câu, thay thế tất cả các URL bằng "URL", thay thế tất cả các số bằng "NUM-BER", hoặc thậm chí thực hiện *quá trình cắt gốc từ* (tức là cắt bỏ phần cuối của từ; có các thư viện Python sẵn có để thực hiện việc này).
   e. Cuối cùng, hãy thử nghiệm một số mô hình phân loại và xem liệu bạn có thể xây dựng được một mô hình phân loại spam hiệu quả, với cả độ nhạy (recall) và độ chính xác (precision) cao hay không.

Các giải pháp cho các bài tập này có sẵn ở cuối sổ ghi chép của chương này, tại *https://homl.info/colab-p*.

7 Bạn có thể sử dụng hàm `shift()` từ mô-đun `scipy.ndimage.interpolation`. Ví dụ: `shift(image, [2, 1], cval=0)` sẽ dịch chuyển hình ảnh xuống hai pixel và sang phải một pixel.